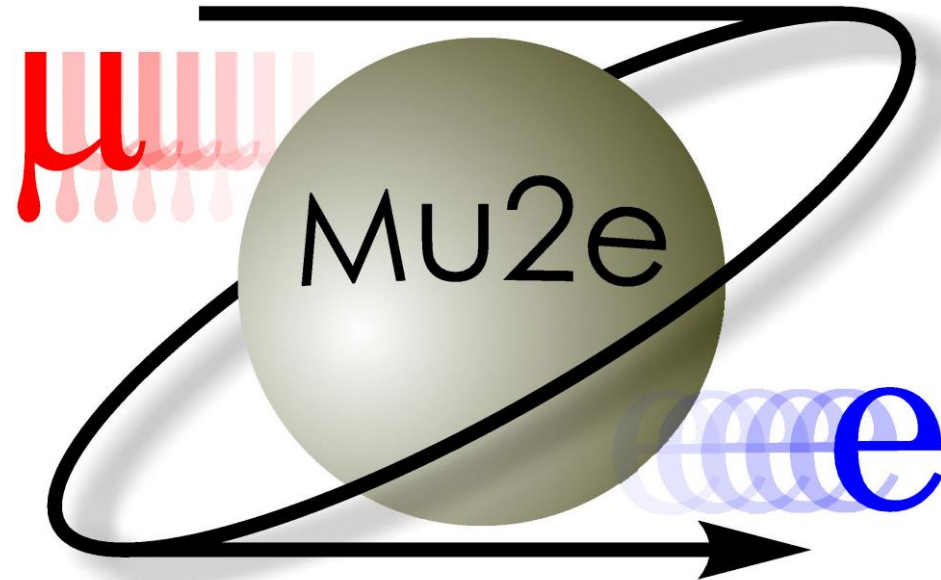


Integrating an XGBoost Model into the Mu2e Offline Framework: From Python to C++



Malik Farouh – City College of CUNY

July 2025-Present



Andrew Edmonds – York College of CUNY



BDT vs. ANN Performance Comparison

Model	AUC	Efficiency	Training Time	Processing New Dataset
ANN1	0.939	48.1%	46.2s	41s
BDT1	0.949	57.2%	1.81s	3.44s

- XGBoost BDT outperforms the current ANN in track-quality selection
- At 99% background rejection, BDT retains ~**9%** more good tracks
- Produces clean momentum resolution across all datasets
- Trains and evaluates faster than ANN
- Ready for large-scale Mu2e production pending TMVA integration

Overview

- **Goal:** Integrate an XGBoost machine learning model into the Mu2e Offline framework for track quality classification
- **Challenge:** Mu2e Offline runs in C++ while machine learning models are trained in Python
- **Solution:** A two-step conversion pipeline bridging Python and C++

Workflow:

- Train XGBoost Model in Python → Convert to C++ with m2cgen → Run Conversion Script → Integrate into Mu2e Offline

Model 2 Code Generator (M2CGen)

- m2cgen (Model 2 Code Generator) is a Python library that converts a trained machine learning model into different programming languages
- Supports multiple output languages including C++, Java, and Python
- Converts the XGBoost model's learned parameters and decision logic into a standalone C++ header file (.hxx)
- The generated C++ file requires **no external dependencies** – it runs on its own
- However, the output is a **generic** C++ file and requires modifications to be compatible with the Mu2e Offline framework

Automating the Conversion of the Generic C++ Header to a Mu2e-Compatible Format

Problem:

- m2cgen produces a generic C++ header that is not directly compatible with Mu2e Offline
- Every time the model is retrained, 4 manual modifications were required

The 4 Changes the Script Makes:

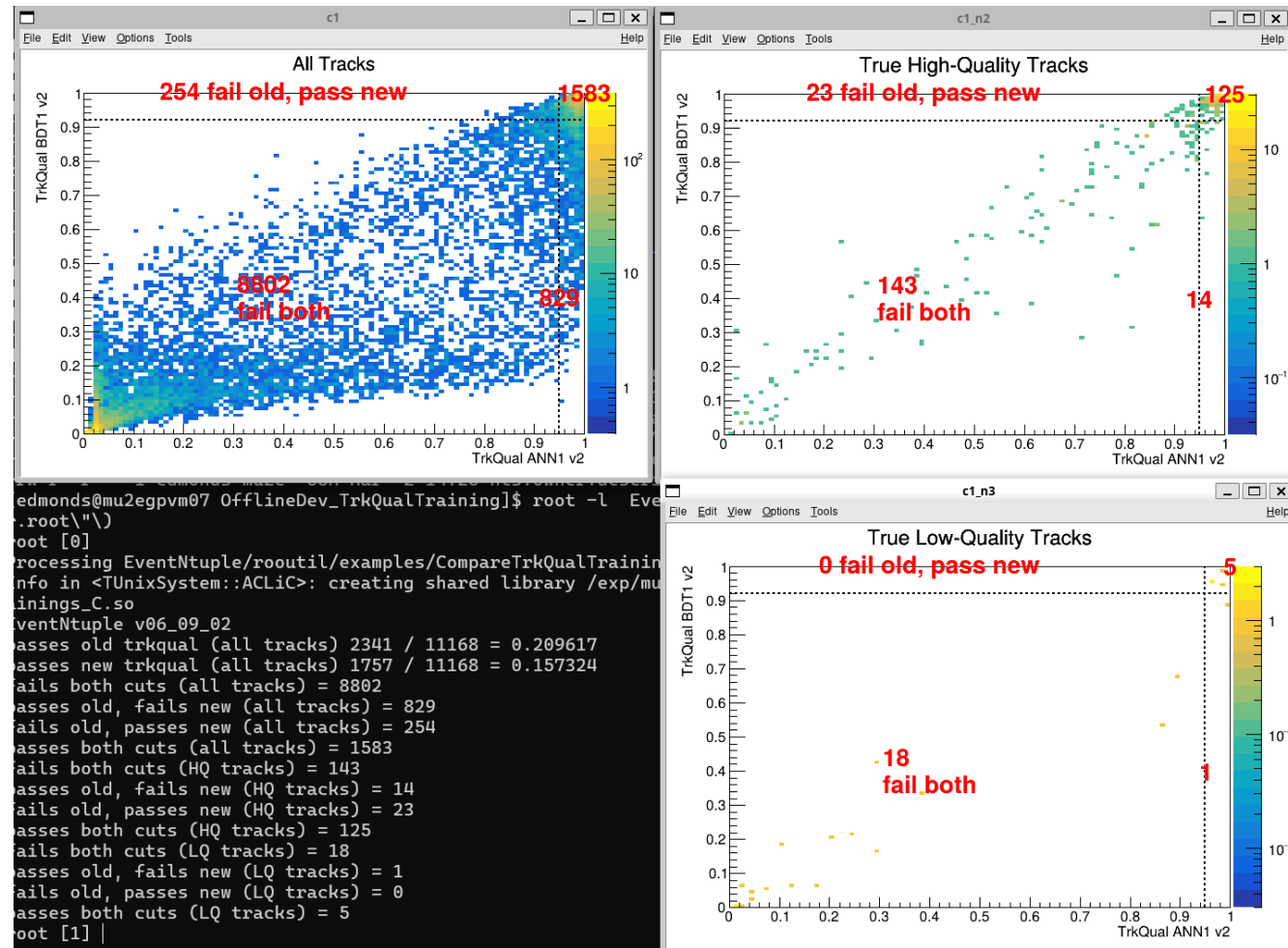
- Wraps the code in a namespace mu2e and class TrkQual_BDT1
- Changes the function signature from double to float
- Comments out (//) the memcpy line: memcpy is a C++ command that copies data from one place to another. m2cgen uses it to write the final result, but Mu2e Offline doesn't like this method.
- Replaces it with direct array assignments and closes the class and namespace

Result:

- The workflow is now fully automated
- Train model → run m2cgen → run conversion script → ready for Mu2e Offline
- No manual C++ editing required on every retrain

BDT vs ANN Performance in Mu2e Offline

- BDT was successfully integrated and tested in the Mu2e Offline framework
- Compared against the existing ANN (Artificial Neural Network) model
- BDT passes **23 true high-quality tracks** that the ANN missed
- BDT loses only **14 tracks** that the ANN passed
- For low-quality tracks, BDT has **0 false passes** vs the ANN
- Results suggest the BDT model performs better overall than the existing ANN



Next Steps

- Current workflow requires recompiling Mu2e Offline every time the model is retrained
- **Goal:** Replace the hardcoded C++ header with a model that reads parameters from an external file at runtime
- The XGBoost model has already been saved as a JSON file (trkQual_model.json)

Two approaches being explored:

- Using XGBoost's native C++ API to load the JSON file directly
- Using Sam Grant's TMVA RBDT method: exports the XGBoost model as a plain text file, which the Mu2e Offline framework can load directly at runtime
- **Next step:** Implement one of these approaches to complete a fully sustainable retraining workflow

Any Questions?



Thank you for your time!